



THINKST
SCAPES

Q3 • 2024

<https://thinkst.com/ts>

Brought to you by



**Most companies find out way too late
that they've been breached.**

Thinkst Canary changes this.

Canaries deploy in under 4 minutes
and require 0 ongoing admin overhead.

They remain silent until they need to chirp,
and then, you receive that single alert.

When.it.matters.

Find out why some of the smartest security teams
in the world swear by Thinkst Canary.

<https://canary.love>

Contents

Introduction	2
Themes covered in this issue	3
Edge cases at scale still matter	4
Flipping Bits: Your Credentials Are Certainly Mine	5
Universal Code Execution by Chaining Messages in Browser Extensions	6
CVE Hunting Made Easy	7
How To Revoke And Replace 400 Million Certificates Without Breaking The Internet	8
Going above and beyond	9
Secrets and Shadows: Leveraging Big Data for Vulnerability Discovery at Scale	10
Eliminating Memory Safety Vulnerabilities at the Source	11
Listen to the Whispers: Web Timing Attacks that Actually Work	12
Secure Shells in Shambles	13
What goes on behind the curtain can be dangerous	14
Breaching AWS Accounts Through Shadow Resources	15
Crashing the Party: Vulnerabilities in RPKI Validation	16
MIFARE Classic: exposing the static encrypted nonce variant... and a few hardware backdoors	17
Fallen Tower of Babel: Rooting Wireless Mesh Networks by Abusing Heterogeneous Control Protocols	18
Attacking Connection Tracking Frameworks as used by Virtual Private Networks	19
MagicDot: A Hacker's Magic Show of Disappearing Dots and Spaces	20
Nifty sundries	21
Can I Hear Your Face? Pervasive Attack on Voice Authentication Systems with a Single Face Image	22
In Wallet We Trust: Bypassing the Digital Wallets Payment Security for Free Shopping	23
Splitting the Email Atom: Exploiting Parsers to Bypass Access Controls	24
6Sense: Internet-Wide IPv6 Scanning and its Security Applications	25
SnailLoad: Anyone on the Internet Can Learn What You're Doing	26
Conclusions	27

Cover photo: Ridgway, Colorado USA. Image by Jacob Torrey (Thinkst)

Introduction

Welcome to the Q3, 2024 edition of ThinkstScapes! This issue focuses on content released, published or presented between the first of July and the end of September, 2024.

Quarter 3 showed a continued uptick in the pace of publications, with a nice mix of marquee and regional conferences taking place, including the “Hacker Summer Camp”. As we’ve seen before, when there are so many great talks, the blogs drop off. A number of top-tier researchers blogged about their conference talks, but included less new work.

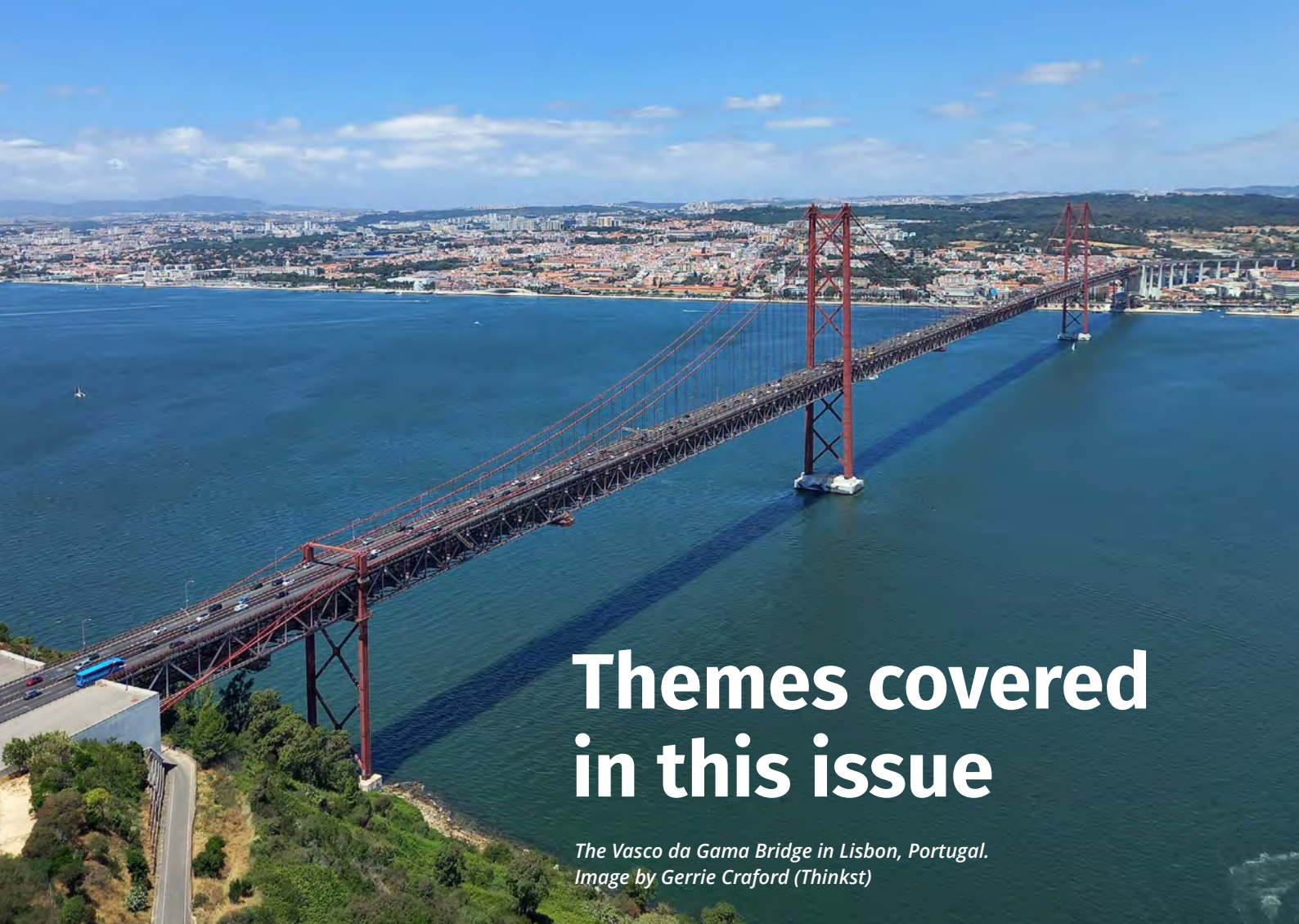
As a reminder: if you are aware of work we’ve missed, a blog post we should have seen or a conference we should have covered, we’d love to hear about it. Please send them to ts@thinkst.com

In addition to almost 1,500 blog posts, this quarter’s content was drawn from the following conference presentations:

Conference Name	Number of talks
Black Hat USA	107
DEFCON 23	101
Digital Forensic Research Workshop	24
BSides Albuquerque	21
BSides Roanoke	13
BSides Brisbane	24
Privacy Enhancing Technologies Symposium	58
BSidesLV	159
HITB	25
Blue Team Con	31
USENIX Security	273
OrangeCon	23
BSides Canberra	56
BSides Tallinn	18
BSides Joburg	19
BSides Bristol	46
SEC-T	26
fwd:cloudsec Europe	13
BSides Denver	10
Off-by-One Conference	17
Pass the SALT	30
European Symposium on Research in Computer Security (ESORICS)	86
The GrrCON Cyber Security Summit and Hacker Conference 2024	66
44CON 2024	11
FUZZING 2024	11
RomHack Conference 2024 Agenda	8
The 27th International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2024)	44
ICML 2024	2778
SummerCon	10
WOOT 24	26
Total	4134



Kruger National Park, South Africa.
Image by Nicholas Rohrbeck (Thinkst)



Themes covered in this issue

*The Vasco da Gama Bridge in Lisbon, Portugal.
Image by Gerrie Craford (Thinkst)*

EDGE CASES AT SCALE STILL MATTER

Works from this theme exploit rarely-occurring issues, but with an internet-wide aperture to end up with impressive results. Look for: mechanising bit-squatting; static code analysis for vulnerabilities across all browser extensions or across web ecosystems; and how Let's Encrypt worries about revoking and reissuing 400M certificates in a week.

GOING ABOVE AND BEYOND

Talks and papers often use state-of-the-art tooling to measure/detect an interesting phenomenon. This theme highlights four works that could have followed that path, but also built robust tooling/research data to help others push the state-of-the-art forward. Look for: large scale collection and remediation of dangling domains and static secret leaks, preventing memory-corruption vulnerabilities across the Android ecosystem, remote timing attack frameworks, and SSH testing at scale.

WHAT GOES ON BEHIND THE CURTAIN CAN BE DANGEROUS

Modern IT systems are composed of many layers. Usually the details at lower levels can be abstracted and safely put out of mind. This theme highlights work that shows that what happens in these oft-ignored places can have significant impacts. See: AWS-internal resources built on your behalf, BGP security weaknesses, stealthy hardware backdoors in access control systems spanning over 15 years, Wi-Fi management plane vulnerabilities, VPN-OS interactions, and a legacy file-system hack in Windows.

NIFTY SUNDRIES

As always, we wanted to showcase work that didn't fit into the major themes of this issue. We cover: bypassing voice authentication with only a picture of the victim's face, racking up bills on locked credit cards, email parsing confusion, scanning IPv6, and a timing attack on remote web clients.



Edge cases at scale still matter

- ✓ *Flipping Bits: Your Credentials Are Certainly Mine*
- ✓ *Universal Code Execution by Chaining Messages in Browser Extensions⁶*
- ✓ *CVE Hunting Made Easy*
- ✓ *How To Revoke And Replace 400 Million Certificates Without Breaking The Internet*

*Heian shrine gardens, Japan.
Image by Alec Badenhorst (Thinkst)*

Flipping Bits: Your Credentials Are Certainly Mine

Author:

Joohei and STÖK



[Code](#)



[Video](#)

In this talk, the pair of researchers dug into mechanising bit-squatting. Bit-squatting is the practice of registering domains that are one bit away from the target (e.g., coogle.com instead of google.com) and waiting for a client to have a transient memory fault during the request flow. Their open-source tool, Certainly, was built from the ground up to handle the DNS requests (automatically fetching SSL certificates on-demand) and the subsequent requests to popular protocols. In addition to the tooling for catching the flipped domains, they also looked at optimising which domains to purchase based on a slight tendency for DRAM to flip from a '1' towards a '0'.

In the lead up to the talk, they registered 25 domains, and had Certainly listening for just under five months. In that time they recorded millions of DNS and HTTP(S) requests, and collected 1000s of credentials and sensitive emails. The collection period corresponded with the large solar storm, and unexpectedly, there was no uptick in bit-flips. As they explored this, they discovered a [paper](#) calling into question the popular cosmic ray explanation of bit-flips.

The talk concludes with a few approaches to mitigate the bit-squatting, from simply buying up the flip domains, to more restrictive certificate pinning to ensure the correct domain and certificate is fetched.

TAKEAWAYS:

- ✓ **The “loot” gathered in less than five months** by collecting on 25 domains shows a significant ROI in the approach. Going the extra steps to find the most valuable squatting domains helps researchers maximise their budget. This technique will probably start trickling down to red teams and pen-testers given sufficient time for an engagement.
- ✓ **Bit-squatting is not a new technique**, and seeing which flipped domains were already purchased shows how attackers are using it. What's important to take away is how putting in the extra effort to make it simple to deploy and handle the collection pays off. Improving a known research topic into a deployable capability pays significant benefits. If these researchers did not release this as open-source, they would have had access to an attack vector that was beyond the rest of the industry for quite some time. Thanks to their generosity, blue teams should now consider this attack vector (as red teams surely will be).

Figure 1. A chart showing the credentials collected over four months and 23 days by listening on 25 domains.



Universal Code Execution by Chaining Messages in Browser Extensions

Author:
Eugene Lim



[Blog](#)



[Video](#)

This work explores how the messaging frameworks used between isolated contexts within the browser can go awry. Extensions can inject a content script that runs from the context of the injected webpage (with some protections), but that script only runs for the lifetime of that page, and with limitations on what extension APIs can be called. Typically there is also a background script, or service worker that acts in concert with the content scripts, but doesn't have direct access to the pages' data or DOM. In order to communicate between these components, there is a messaging API to send and receive messages.

The researcher identified that the API provides little guarantee that the messages received from a tab or page came from the content script, instead of JS on the page itself. Most extensions don't implement additional authentication checks, so a malicious page's JS can send messages to the background service. Continuing in the vein of browser messaging, there are APIs for background scripts to communicate with native processes over stdio pipes. The example provided was a PKI smartcard application library that could perform smartcard operations on a webpage if the native library and extension was installed. The background script in this extension simply relayed messages between the page and native component, which had a vulnerability that would load a DLL at a user-provided path.

While this chain appears fragile and of limited impact, the researcher then used SemGrep to identify other extensions with a large user population with similar designs: background scripts acting as relays to native components that listen for content script messages from any origin. This ability to scale a vulnerability discovery into multiple related issues highlights how powerful static analysis can be as an attacker.

TAKEAWAYS:

- ✓ **This work shows how impact can be scaled up** through automation. Looking across the entire extension store and performing semantic searches allows for rapidly finding vulnerable extensions and understanding impact.
- ✓ **We'll continue to see how packages** and extensions that "only" have 100k users have bugs – but now it's easier to find more instances of the same issue in other locations.

Figure 2. A high-level diagram of the three interconnected components that allows for loading a DLL from JavaScript.



CVE Hunting Made Easy

Author:
Eddie Zhang



[Blog](#)



[Code](#)

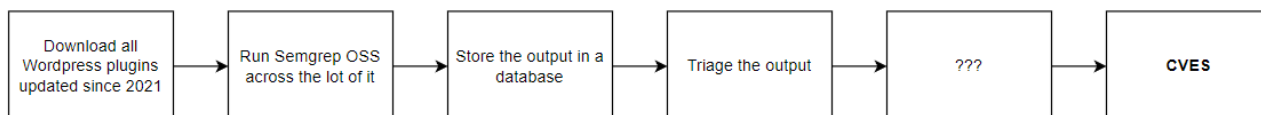
This blog looked at how to pull together a pipeline to analyse open-source WordPress plugins (as an example) to reduce the effort needed to find exploitable vulnerabilities. In three afternoons, the author reported and was issued 14 CVEs for a number of bug classes in plugins with small to moderate install bases. Using only existing tooling, the author scripted: downloading all plugin packages, running the off-the-shelf Semgrep checks for PHP, and putting all the results into a database. Then, spending no more than five minutes per finding, the author would triage them as either potentially exploitable or a false positive.

Of the findings triaged as worth a deeper look, the authors gave themselves a further 15 minutes to build a proof-of-concept exploit. Again they were helped by existing testing frameworks for WordPress, minimising the overhead time to build an environment to perform manual dynamic analysis for exploitation. Despite the self-imposed limitations, they were able to find a fair number of vulnerabilities, but note that there are likely many more waiting to be found.

TAKEAWAYS:

- ✓ **None of these bugs were particularly novel**, but the ease with which a pipeline was built to find these at-scale by an individual warrants being highlighted. The level-of-effort to find over a dozen CVEs using existing tooling should highlight both that it's well within the scope of most attackers, and that there's little reason a similar approach shouldn't be in place on the development side. While CodeQL does integrate with GitHub, and could provide a higher level of confidence in its findings with bespoke queries, this blog shows that off-the-shelf checks find real bugs. With improvements in disassembly and decompiler technologies, it won't be long before these pipelines will exist for closed-source applications as well.
- ✓ **By self-imposing time limits** on triage and exploitation, this blog shows how a broad approach can net results. We generally think of bug finding as a very deep search into a particular target, whereas a broad approach without a specific target predetermined is also worthwhile.
- ✓ **Most of the plugins that the researcher found** vulnerabilities in were small, with a few hundred to at most ~40,000 installs. If you plan on hosting WordPress, it's probably best to stick to the most popular plugins that have likely had the most thorough security reviews.

Figure 3. A high-level flowchart of the pipeline to find exploitable vulnerabilities in WordPress plugins.



How To Revoke And Replace 400 Million Certificates Without Breaking The Internet

Author:

Aaron Gable



[Slides](#)



[Video](#)

This talk was instigated by a Let's Encrypt mass revocation event. There have been a few instances where a portion of issued certificates had to be revoked and reissued in a five-day window. While those past events were a small percentage of the total issued certificates at the time, the speaker explored what would happen if Let's Encrypt had to reissue all ~400M of their issued certificates. As shown in the figure, a back-of-the-envelope calculation paints a bleak picture. This timeframe could be reduced by upgrading the HSMs performing the signing operations, but it's unlikely to achieve the advertised 10,000 signatures per second in operation.

In order to address these issues, Let's Encrypt is working on three fronts to both reduce the signature volume needed and help clients get reissued certificates faster. While the rules on certificate authorities (CAs) dictate revocation in five days or less, the consequences of revoking over 50% of the internet's certificates without reissuing them would be severe. Machine-to-machine connections would likely cease to work, and end-users would end up learning how to bypass TLS connection warnings, opening the door for fraud down the line.

The original architecture for the OCSP revocation system developed by Let's Encrypt pre-signed the revocation statuses every four days, and then provided the static data to requesting clients. This reduced complexity and made the signature throughput predictable, but there are many certificates that are never checked. The new architecture developed to help reduce the signature load during a mass revocation process is to sign the OCSP data live upon request, and then cache that response. This means that test certificates, or internally-used certificates that are not validated generate no signature load.

Figure 4. The naive maths for how long it would take to revoke 400M certificates in an urgent scenario. The rule for CAs requires full revocation and reissue in less than five days.

The other two improvements are to make reissuing certificates faster and to guide clients towards shorter-lived certificates. By making clients smarter about checking for the need to reissue, the window between revocation and reissue can be shortened. Going one step further, if Let's Encrypt can start issuing certificates with even shorter lifespans to a bulk of their clients, then the window where things may be broken shrinks more.



TAKEAWAYS:

- ✓ **It's pretty amazing** what can be accomplished with a small team and both a dedication to automation and foresight to start preparing for future issues.
- ✓ **More than half the internet** uses Let's Encrypt, which has a staff of a dozen. Thinking about how issues unfold at larger and larger scales is a valuable process, and one worth highlighting.



Going above and beyond

- ✓ *Secrets and Shadows: Leveraging Big Data for Vulnerability Discovery at Scale*
- ✓ *Eliminating Memory Safety Vulnerabilities at the Source*
- ✓ *Listen to the Whispers: Web Timing Attacks that Actually Work*
- ✓ *Secure Shells in Shambles*

*Ridgway Colorado.
Image by Jacob Torrey (Thinkst)*

Secrets and Shadows: Leveraging Big Data for Vulnerability Discovery at Scale

Author:
Bill Demirkapi



This research explored two well-understood vulnerabilities at scale: dangling domains and static secret disclosures. Dangling domains exist when a domain (or sub-domain) points to an IP or service that is only temporarily in control by the domain owner. After the resource it points to is released (either the cloud instance and IP terminated, or the shared service deactivated), if the DNS entry is still there, another entity can try to claim the underlying resource. A classic example of this is a cloud instance pointed to by a company's domain name, where the organisation forgets to remove the DNS entry after the cloud instance is destroyed. Then an attacker can quickly spin up and down instances until one is issued with the same IP address.

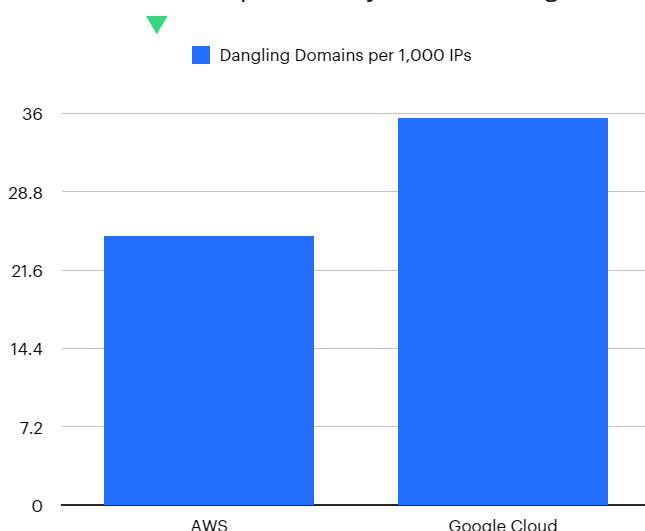
Static secret disclosure is when an API key or other secret is inadvertently uploaded publicly. The author notes that GitHub has secret scanning for some types of credentials pushed to a repository. However, beyond a few of these common places of inadvertent disclosure, there wasn't a larger body of research looking for these leaks.

With these two vulnerability classes in mind, the author then looked at how to scale up the discovery process. Reversing the typical approach of having a target organisation and then searching for vulnerabilities in their online footprint was a key insight. In this model, scalable systems were built that looked for as many instances of dangling domains and secret disclosures as possible, then later analysis could determine the impacted organisation. The first step for dangling domains was to enumerate as many available IPs by cloud provider as possible, and then cross reference them with passive DNS records to see what domains point to those IPs. Cloud providers have worked to reduce this weakness by limiting the number of IPs available from the pool to each account; the researcher had to create many accounts to maximise coverage. A number of high-profile organisations were discovered to have dangling domains, these were reported to that organisation for remediation.

For static secrets, the researcher wanted as large a dataset as possible, so they looked to malware collection frameworks, such as VirusTotal. VirusTotal allows for users to submit YARA rules (i.e., regular expressions) and get all samples that match. As a large variety of file types are submitted to malware analysis frameworks, the author was able to analyse ~5 million input files, discovering over 15,000 valid credentials. The most common leak vector came from Android applications that were uploaded for analysis, many of which had static credentials stored within.

Finally, the author pushed the discovered credentials to GitHub as private gists. This would trigger GitHub's secret detection and revocation processing, allowing the author to prevent anyone from using the secrets maliciously.

Figure 5. A chart showing how many domains were dangling per 1,000 IPs for AWS and GCP.



TAKEAWAYS:

- ✓ **Going beyond the specific findings**, this work showed what a cloud-native attacker mindset could do when scale was a first-class consideration.
- ✓ **There have been a few instances** where bug bounties have shown their inability to handle scalable bug classes; this work shows that it's only a matter of time until that is the norm.
- ✓ **Expect to see bounty hunters** looking for a bug class at scale, and then looking at instances of that class in scope of a bounty program instead of starting from the bounty scope.

Eliminating Memory Safety Vulnerabilities at the Source

Authors:

Jeff Vander Stoep
and Alex Rebert



Blog

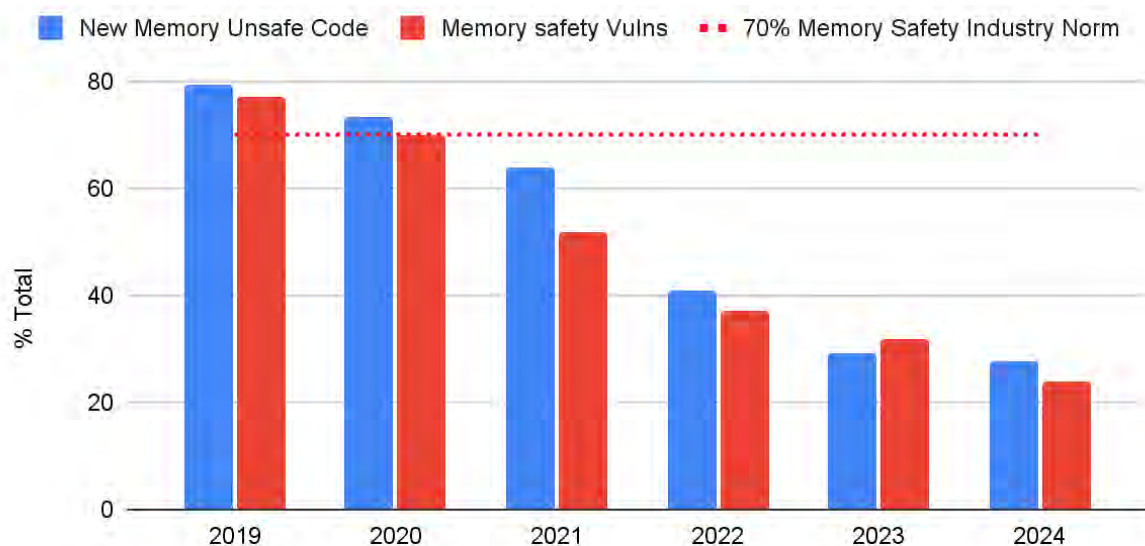
This blog reviewed the Android source code and associated vulnerabilities over the last five years as there has been a concerted effort to switch to memory-safe programming languages. For Android, memory-unsafe code is only changed to fix bugs – all new features are built in a memory-safe language. When looking at memory-safety vulnerabilities, they found that even with the slight increase in new unsafe code, the number of relevant bugs has continued to decrease. This data supports the claim that, as code bases mature, the newest code is more likely to hold vulnerabilities – code that's been looked at or analysed for longer shows an exponential decrease in bug density. This decay has allowed Google to focus on adding new features in memory-safe languages (and improving interfaces between utilised languages) instead of porting existing code to a safer language.

The blog also includes the evolution of Google's code improvement processes. This shows the progression from simply being responsive to reported bugs, to proactively fuzzing for bugs, to enforcing static checks on code properties. A couple of side-benefits of statically-verified safety properties are showcased, such as a 95% performance increase when functionality can be removed from a sandbox.

TAKEAWAYS:

- ✓ **The empirical validation** of how vulnerabilities-per-code quantity decays over time is helpful in planning how to invest development and modernisation time.
- ✓ **The obvious answer** of rewriting all code in a memory-safe language may not result in an overall improved security posture, whereas ceasing new development in memory-unsafe code (and improved static and dynamic analyses) can result in industry-beating outcomes.
- ✓ **Keep in mind** that Google has the security investments and the mature analysis programs in place to find and fix vulnerabilities at scale. They may obtain better results than the average organisation.

Figure 6. A chart showing how, despite more memory-unsafe code being present in the Android codebase, the amount of memory-safety vulnerabilities have continued to decrease well below industry norms.



Listen to the Whispers: Web Timing Attacks that Actually Work

Authors:
James Kettle

 [Slides](#)

 [Paper](#)

 [Code](#)

Figure 7. An example of the sources of noise that can prevent a timing attack from working against a network target, especially in real-world situations



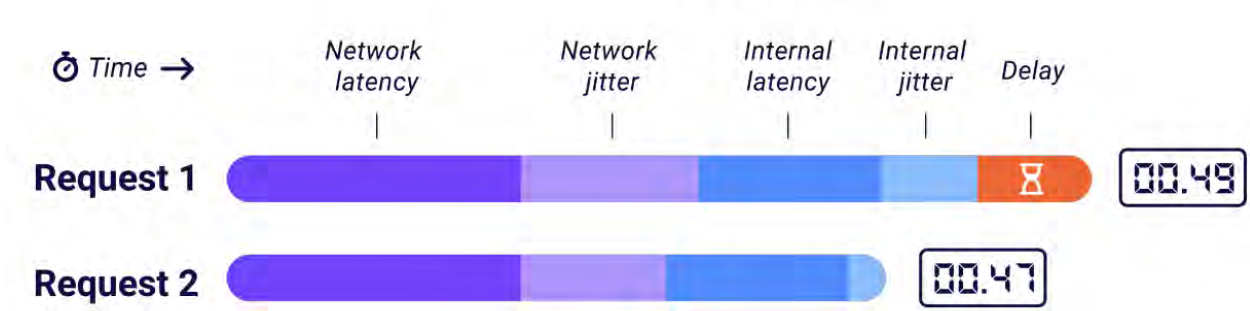
This work matured this researcher’s previous work into remote timing attacks. Timing attacks have been around as a side-channel for a long time, but very small timing differences are usually hidden in the network noise. Last year, this researcher showed how to perform remote timing attacks against HTTP/2-compatible servers while removing the network latency. These “single packet attacks” (SPAs) worked by combining multiple requests into a single packet, so responses would come in order of request completion, removing network jitter. This 2024 work improves upon this by sending two partial requests in a single packet, then the final bytes of those requests in a second packet, to minimise the impacts of TLS decryption time.

The research then explores the classes of vulnerability that were discoverable using timing differences, from cache poisoning to hidden parameters, etc. The research has been integrated into multiple open-source tools, and is being used to find weaknesses across multiple web server implementations.

TAKEAWAYS:

- ✓ **The improved single-packet-attack** opens up the door to detecting multiple classes of timing incongruities. Expect to see a lot more timing-discovered vulnerabilities as these tools become more mainstream for bug bounty hunters.
- ✓ **Changes and performance improvements** at one layer in the stack can have security-relevant impacts at other layers.
- ✓ **This is yet another demonstration** that secure composition is an open challenge, and whenever there are multiple layers changing without a comprehensive understanding, vulnerabilities will crop up.

$$\text{success} = \frac{\text{signal}}{\text{noise}}$$



Secure Shells in Shambles

Authors:

HD Moore and
Rob King

-  [Slides](#)
-  [Code](#)
-  [Video](#)

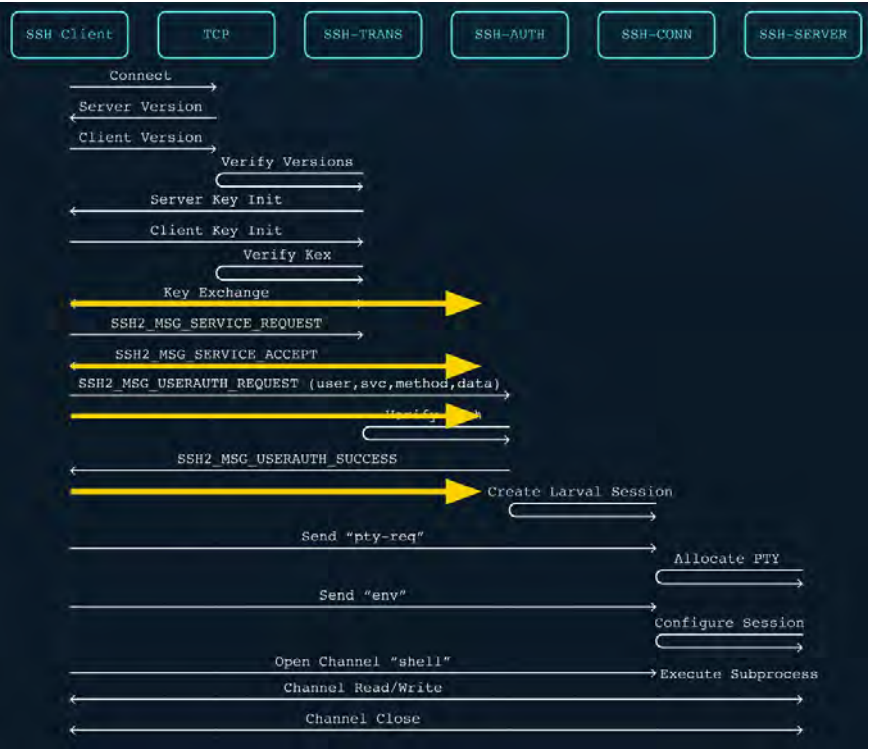
This research built a tool, SSHamble, to explore different SSH implementations from a security perspective. The researchers note that oftentimes, SSH is treated as a single implementation, instead of a complex protocol with multiple implementations, forks, and authentication integrations. By looking at past bugs in SSH implementations as a guide for deeper exploration with SSHamble, they find a number of new vulnerabilities. One such example is shown in the figure below. There was a past vulnerability where trying to jump forward in the state machine by simply requesting a SSH session worked from a certain pre-authentication state. SSHamble attempts to request a session from all stages, finding such a vulnerability in a network appliance.

In addition to flaws in specific software implementations, the research explores different authentication schemes, and how in general, authentication that occurs after a session is established is likely to be vulnerable. These weaknesses ranged from environment variable injection to forwarding traffic to other hosts on the server's internal network. Finally, there is a review of the privacy weaknesses of public key SSH authentication and suggested deployments to limit those risks.

TAKEAWAYS:

- ✓ **As noted in this talk**, SSH is often considered a singular entity, and too often are the many implementations and forks lumped together.
- ✓ **Much like we've seen** (and highlighted in previous ThinkstScapes) with the security concerns of non-browser-based TLS, lesser-used SSH implementations are likely to have high-impact bugs.
- ✓ **With tooling like SSHamble now released**, expect to see more high-consequence bugs found in niche appliances.

Figure 8. A figure showing the SSH connection state machine, and places (yellow arrows) where sending a request for a session can jump past authentication depending on the implementation.





What goes on behind the curtain can be dangerous

- ✓ *Breaching AWS Accounts Through Shadow Resources*
- ✓ *Crashing the Party: Vulnerabilities in RPKI Validation*
- ✓ *MIFARE Classic: exposing the static encrypted nonce variant...
and a few hardware backdoors*
- ✓ *Fallen Tower of Babel: Rooting Wireless Mesh Networks
by Abusing Heterogeneous Control Protocols*
- ✓ *Attacking Connection Tracking Frameworks as used
by Virtual Private Networks*
- ✓ *MagicDot: A Hacker's Magic Show of Disappearing
Dots and Spaces*

*Lan Ha Bay, Vietnam.
Image by Daniel de Villiers (Thinkst)*

Breaching AWS Accounts Through Shadow Resources

Authors:

Yakir Kadkoda,
Michael Katchinskiy,
and Ofek Itach



Slides



Code

This work explored how to exploit predictable naming of S3 buckets by other AWS services. S3 offers a global namespace (per partition), so name collisions are more likely, and can be adversarially created. Many AWS services automatically create one or more S3 buckets to store sensitive metadata for that service to use, and in many cases don't handle collisions well. If an attacker can create a bucket that these services would use, either the service creation will fail, or if the permissions on the attacker's bucket allow for external access, the service will use the attacker's bucket. Then, depending on the service and the types of data stored in the bucket, attackers can create further impacts against the victim up to and including a full account takeover by injecting IAM roles into the victim's account.

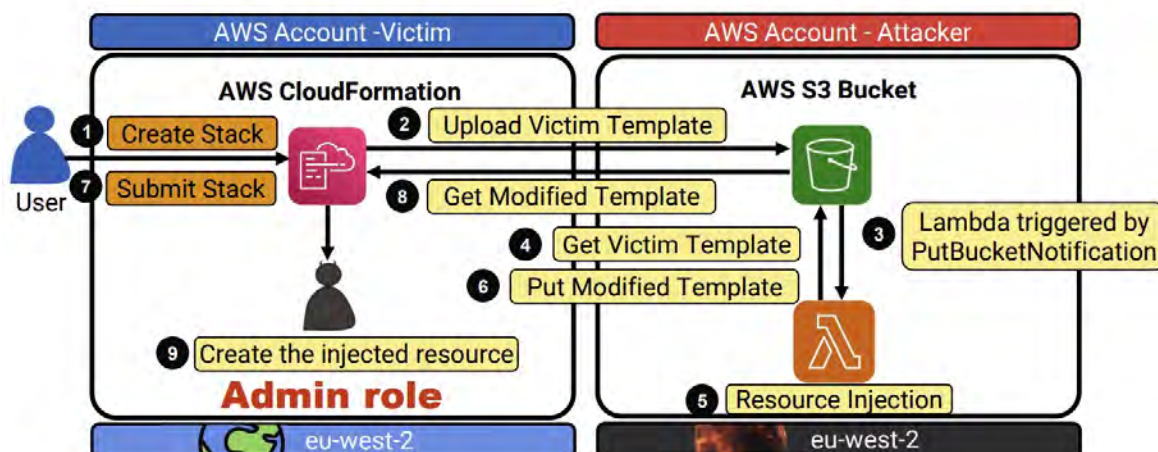
By examining a large number of services and their bucket naming schemes, many were found to use the account ID as the unique key. Account IDs are not considered secrets, thus are commonly found in code repositories, or shared more widely. By finding an account ID, the attacker could pre-generate S3 buckets with writable permissions, and wait until one of the vulnerable services was employed—then continue their attack.

TAKEAWAYS:

- ✓ **The single namespace** of S3 has created a host of problems over the years. There is more nuanced error handling needed for collisions in a multi-tenant environment, and even multiple teams within AWS cannot get that right.
- ✓ **Expect to see more instances** of name collision vulnerabilities across all cloud services with a global namespace.
- ✓ **As shown in other featured research**, each *aaS vendor has a slightly different way to handle service accounts/access, for both internal and external actors. When one tool creates resources or accounts in the background, there should be more clear visibility into those resources, and how to monitor their behaviours.

Figure 9. A flow diagram showing how an attacker can inject a privileged role into a victim's account by pre-generating predictable auto-created bucket names.

CloudFormation: Full Attack Scenario



Crashing the Party: Vulnerabilities in RPKI Validation

Authors:

Niklas Vogel,
Donika Mirdita,
Haya Schulmann, and
Michael Waidner



[Slides](#)



[Paper](#)

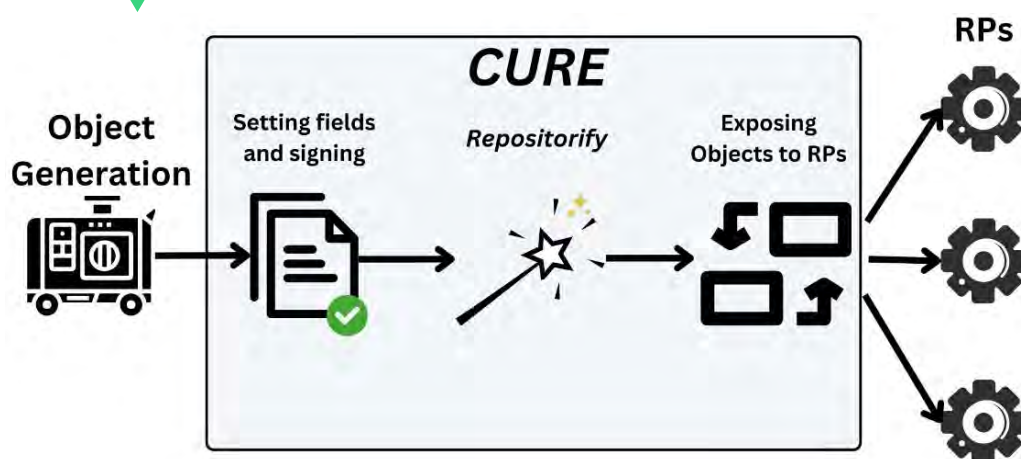
This work explored the security features added onto the BGP routing protocol. These features, known as RPKI, allow for the owners of IP ranges to sign the advertisements of those ranges, thereby reducing the risk of a malicious ISP from hijacking the routes to those IPs. While BGP dates back to the early days of the internet, RPKI is a relatively new addition, only released in 2012. The way that the RPKI protocol was added to BGP means that disrupting the relaying of signed ownership information causes the routers to revert to the old, unauthenticated scheme. In other words, a DoS attack on the RPKI relays opens up routes to being hijacked by malicious route advertisements.

The researchers had to build a custom fuzzing system, CURE, which created ownership objects that were then sent to the RPKI relaying services to observe their behaviour. Both random mutation, and structure-aware object generation techniques were used, the latter uncovering slightly more bugs. In total, 11 issues were disclosed, but the researchers note that deploying fixes to these systems will take quite a while—even today, 12 years after RPKI was released, over 40% of BGP routers still don't perform any validation on routes being advertised

TAKEAWAYS:

- ✓ **BGP is a core network protocol** that for some reason seems to get less attention than others, such as HTTP, or DNS.
- ✓ **It's nice to see that there are attempts** to improve the security of the system against hijacking, but whenever there is security bolted-on later, there is always a risk of failing back to insecurity. As shown in this work, availability goals insist that if there are RPKI issues (from e.g., a DoS), the security guarantees will be disabled and the protocol will fall back to no integrity.
- ✓ **While there has been a push to** improve the deployment of RPKI across the core routers of the internet, over a decade after the RPKI RFC was released, only slightly more than half the routers try to support it.
- ✓ **As cyber attacks escalate**, expect to see these vulnerabilities exploited in concert with route hijacks.

Figure 10. A high-level diagram of the custom fuzzing pipeline developed for RPKI testing.



MIFARE Classic: exposing the static encrypted nonce variant... and a few hardware backdoors

Author:

Philippe Teuwen



[Blog](#)



[Paper](#)



[Code](#)

This research explored the newer variants of MIFARE proximity RFID cards to evaluate their defences against card-only attacks. The MIFARE protection scheme is generally considered broken if interactions between a card and reader can be recorded – card-only attacks weaken the system even if only the card can be physically interacted with. The -S version of the card, made by Shanghai Fudan Microelectronics, was designed with fixes for these card-only attacks.

The newer version of the card offers a different nonce generation scheme that uses data from the card's pages that are not readable without a key. However, in fuzzing the protocol to see if the card would respond in unexpected ways, the researchers found that there was a response to an (officially) invalid command. Digging in, they discovered that there is another key hard-coded into the cards by the manufacturer. This key (shown in the figure) is static across every card, an attacker with knowledge of this key could weaken the nonce generation, allowing them to clone a keycard with only two seconds of physical proximity. The attack does rely on access to a reader, but separately at a later date.

Exploring other variants of the MIFARE compatible cards, the researcher found that the non-hardened versions also have a (different) backdoor key. Looking across a number of collected cards, they could verify the same key present dating back to as early as 1997. The key was present across different vendors and fabrication facilities.

TAKEAWAYS:

- ✓ **While MIFARE cards have been** considered insecure for quite some time, it's particularly insidious to add a hardware backdoor to a variant that improves the security.
- ✓ **For a product designed** as an access control, inserting hardware backdoors into all products raises serious concerns.
- ✓ **The earliest card** with a detected backdoor dates back to the 20th century. That the backdoor is only coming to light now, over 25 years later shows how ill-prepared we are for malicious hardware coming from the factory.
- ✓ **The XZ software** backdoor attempt caught earlier this year set the internet on fire. This was a successful deployment of a backdoor for a quarter century, with much less fanfare.

Figure 11. The recovered backdoor key to authenticate to all of the "improved" MIFARE compatible cards.

A396EFA4E24F
FM11RF08S universal backdoor key

Fallen Tower of Babel: Rooting Wireless Mesh Networks by Abusing Heterogeneous Control Protocols

Author:

Xin'an Zhou,
Zhiyun Qian, Juefei Pu,
Qing Deng, Srikanth
Krishnamurthy, and
Keyu Man



[Slides](#)



[Paper](#)



[Code](#)

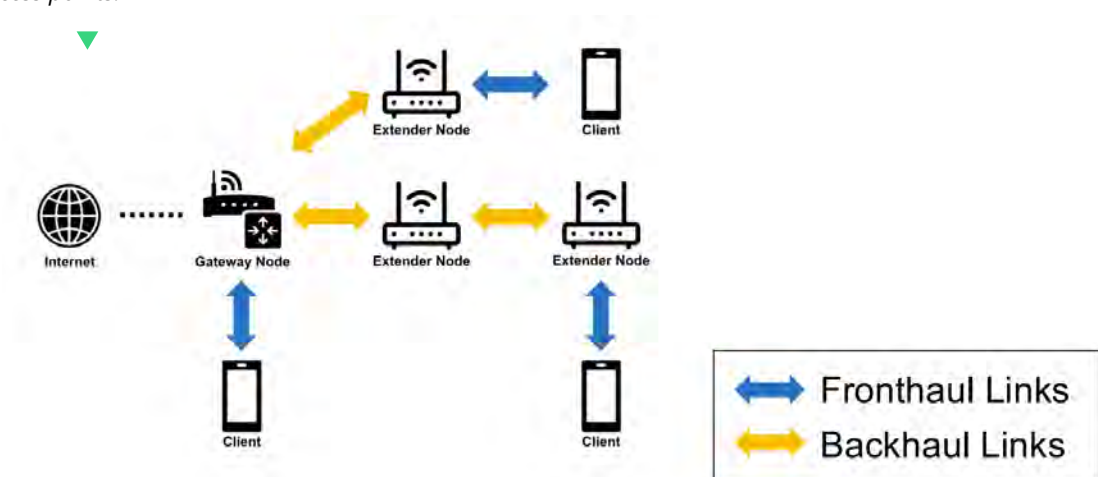
This research explored the protocols used by wireless mesh networks to communicate security-sensitive configuration information from the core router to the end access points. This type of protocol, referred to as Network Access Policy Synchronisation (NAPS), is how Wi-Fi passwords are set, firewall rules shared, etc. While there has been a push for a standard to allow for inter-vendor interoperability, each vendor has their own NAPS protocol as well. The researchers found that unlike the core Wi-Fi security protocols, such as WPA2 which undergo significant review, these protocols are homemade and generally insecure. This research assumed that the attacker was able to connect to the Wi-Fi network as a standard client, from there they were able to either reveal all the sensitive configuration data, or on four out of the seven vendors, obtain a root shell on the core router.

When looking at the implementations from multiple vendors, the research found two classes of weakness: either no cross-layer trust between nodes, or breakable cross-layer trust. The EasyMesh standard uses in-band communication, and performs no authentication; any client that sends a request and a public key will receive the configuration data. These findings have been communicated to their respective vendors, and many have patches available

TAKEAWAYS:

- ✓ **When thinking about wireless networks**, the main security aspects considered are the encryption and authentication of the fronthaul links (e.g., WPA, RADIUS, etc.). This research shows not only that there are other protocols at play, but that each vendor has chosen a different way to implement the mesh functionality.
- ✓ **There are certainly more of these** augmentation protocols that are vendor-created and likely to be lacking in security scrutiny – expect more similar discoveries.
- ✓ **We tend to treat our networks** as set and forget. This research is a good reminder to take the 10 minutes to update the firmware of your routers, and occasionally change the passwords for guest networks.

Figure 12. A network diagram showing how mesh networks use two classes of links for communicating with clients and of mesh access points.



Attacking Connection Tracking Frameworks as used by Virtual Private Networks

Authors:

Benjamin Mixon-
Baca, Jeffrey Knockel,
Diwen Xue,
Deepak Kapur,
Roya Ensafi, and
Jed Crandall



Paper

This research explored how the connection tracking and NAT OS components in VPN servers can result in VPN clients being vulnerable to co-resident VPN-using attackers. While the VPN software isolates clients' traffic, the host OS's NAT must use a finite number of ports to deconflict connections to the same host. This deconfliction results in a shared resource that can be abused by an attacker, exhausting port allocation. This attack assumes that the attacker is able to determine the target's public IP. The first step in the attack, which subsequent effects rely on, is for the attacker to establish a connection with the VPN server prior to the target. Then the attacker sends traffic to the target's public IP for many ephemeral ports with a source port of the VPN server's listening port, this will cache a mapping from the target to the attacker in the VPN's connection tracking cache. When the target tries to connect to the VPN, the request will be forwarded to the attacker, who can create a second VPN connection to relay the target's traffic. The target's traffic will be encrypted, but heuristics and metadata about packet sizes, etc. will be visible to the attacker.

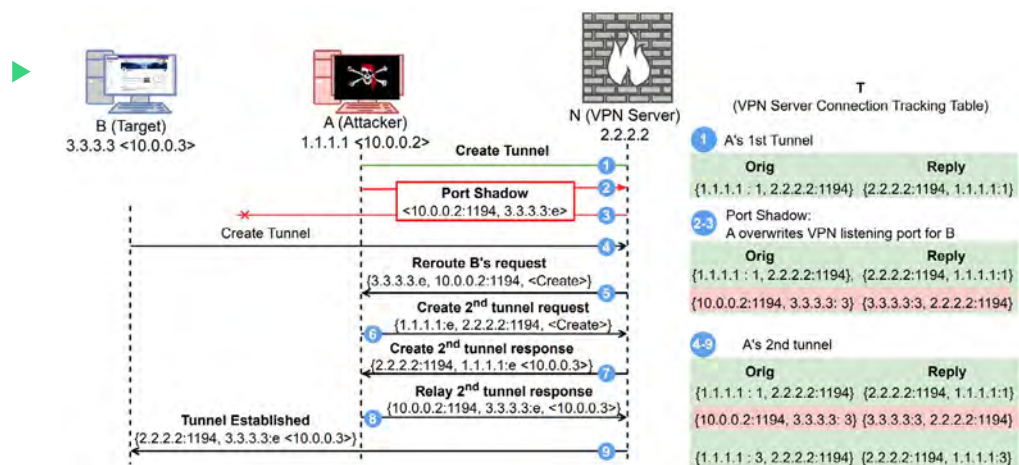
From this primitive, which the researchers call a Port Shadow, the paper then details how this can be used to: inject DNS responses to get the victim to connect to the attacker, create a DoS for the victim, and even port scan/open external ports to the victim through the VPN. The defect was validated on multiple OS, VPNs, connection tracking components, and production VPN offerings. While all Linux OSes were impacted, FreeBSD supports multiple connection tracking solutions, half of which were immune.

Finally the researchers formally modelled the connection tracking interactions with shared VPN users to validate they had enumerated all possible protocol-level issues. They recommend a number of mitigation strategies to prevent both the root-cause Port Shadowing as well as the follow-on weaknesses.

TAKEAWAYS:

- ✓ **These types of shared resource** collisions have been seen on local NAT networks (e.g, coffee shop Wi-Fi networks). That they open users of VPN software to new classes of attack highlights that there is risk in such shared (security) services.
- ✓ **Formal modelling of security-critical** components shows that automatically discovering these types of composition vulnerabilities is feasible.
- ✓ **In addition to promoting** open-source security reviews, a related push for model-checking protocols would offer large benefits beyond implementation flaws.

Figure 13. A diagram showing the steps for an attacker (A) to inject themselves into the communication path between the target (B) and VPN server (N) that both A & B have access to. The table on the right shows how the port conflict created by the Port Shadow causes the traffic to be routed through A.



MagicDot: A Hacker’s Magic Show of Disappearing Dots and Spaces

Author:
Or Yair

 [Slides](#)

 [Blog](#)

 [Video](#)

 [Code](#)

This research started off with a discovery that Windows treats files and folders ending with spaces or periods in a strange manner. While DOS paths allow for trailing spaces and periods, the NT APIs strip them out. After finding documentation from Microsoft to avoid ending file/folder names with these characters, the researcher explored how to use the duality of filesystem views to impact security. When a file with a trailing dot was in the same folder as a file with the same name, but without the dot, actions on the trailing-dot file were performed on the dotless file.

This allowed for a few tricks based on this primitive: ways to hide processes, ways to modify files without the requisite permissions, and a powerful RCE attack. The latter (now fixed) occurred when an archive contained a symbolic link to a destination outside of the archive, and then a regular file with the same name as the link, with a trailing dot. When the Explorer extraction process begins, it writes the link into the folder where the archive is being extracted. The trailing-dot file is then written onto the link, which means it writes to the link’s destination. By setting the link to point to e.g., the Startup Items folder, simply extracting an archive would write arbitrary code to be run on next boot.

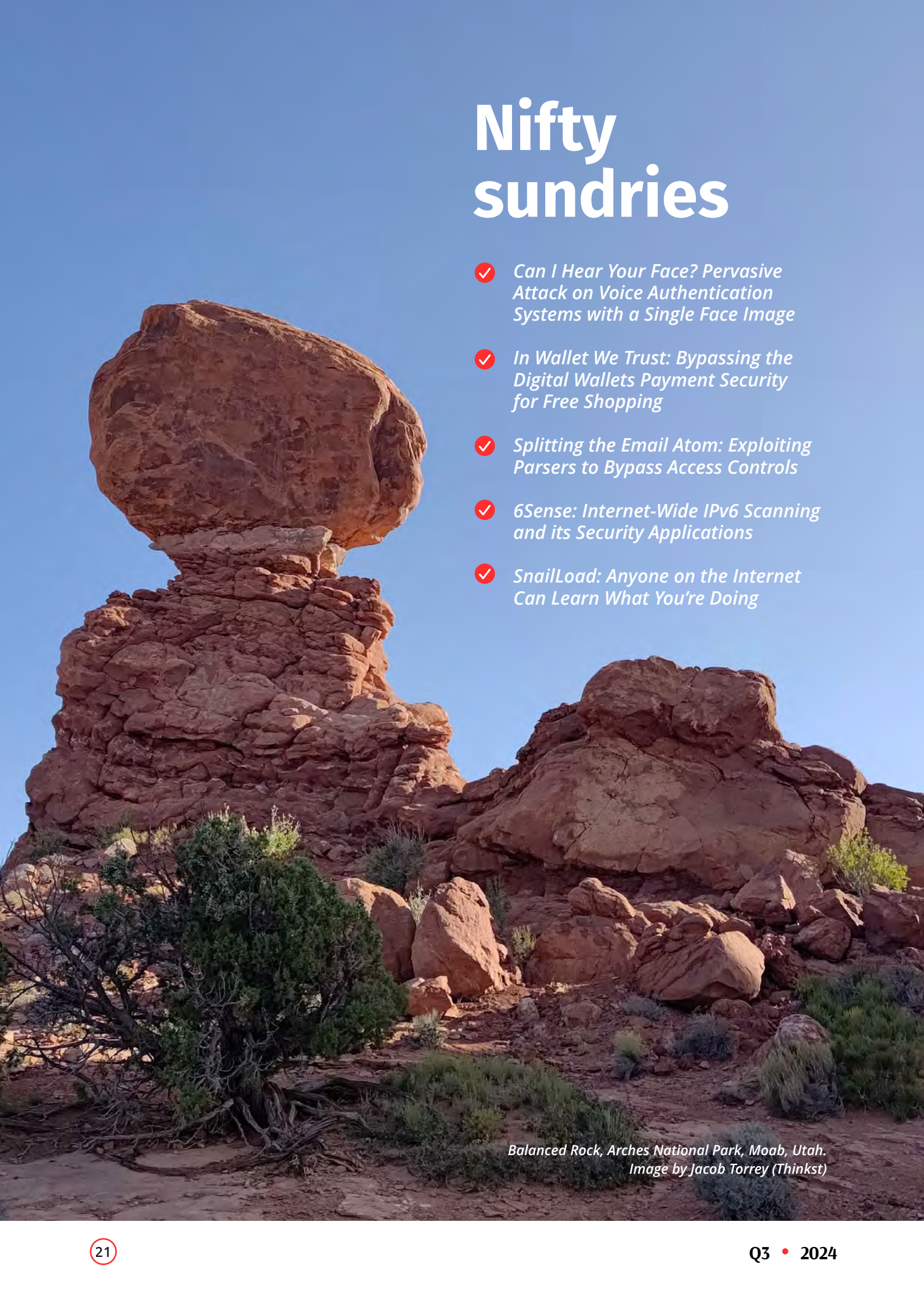
TAKEAWAYS:

- ✓ **As is usually the case** with backwards compatibility, bending over backwards to support decades-old software can result in security issues.
- ✓ **That the brand new archive** extraction capabilities (which shouldn't have to pay lip service to pre-existing software) got bitten by this bug class is concerning. When the defense in place to avoid file naming issues is just a paragraph in the documentation, things will go wrong.
- ✓ **While Microsoft has fixed these** specific symptoms, expect to see more issues crop up with the same root cause.

Figure 14. A table showing some examples of how DOS paths are resolved by the NT subsystem: the crux of this research.



DOS Path	NT Path (MagicDot)
C:\example\example.	\\?\C:\example\example
C:\example\example...	\\?\C:\example\example
C:\example\example<space>	\\?\C:\example\example
C:\example\example<space><space>	\\?\C:\example\example
C:\example.\example	\\?\C:\example\example
C:\example<space>\example	\\?\C:\example<space>\example



Nifty sundries

- ✓ *Can I Hear Your Face? Pervasive Attack on Voice Authentication Systems with a Single Face Image*
- ✓ *In Wallet We Trust: Bypassing the Digital Wallets Payment Security for Free Shopping*
- ✓ *Splitting the Email Atom: Exploiting Parsers to Bypass Access Controls*
- ✓ *6Sense: Internet-Wide IPv6 Scanning and its Security Applications*
- ✓ *SnailLoad: Anyone on the Internet Can Learn What You're Doing*

*Balanced Rock, Arches National Park, Moab, Utah.
Image by Jacob Torrey (Thinkst)*

Can I Hear Your Face? Pervasive Attack on Voice Authentication Systems with a Single Face Image

Authors:

Nan Jiang, Bangjie Sun,
Terence Sim, and
Jun Han



Paper



Code

This work looked at bypassing voice authentication systems. While there has been considerable work in using recorded clips of the victim to “deepfake” their voice, this research explored synthesising voice prints with only an image of the target. The researchers trained a model on a number of face images and voice samples, and their model learned the visible aspects of a face that impacted voice (e.g., gender, age, lip size, etc.), which was then used to try and defeat voice authentication systems. The model would then output a distribution of audio samples of the cloned voice saying the required phrase. Each sample would allow for exploring variations on the voice that were related to invisible attributes, such as voice box size and composition.

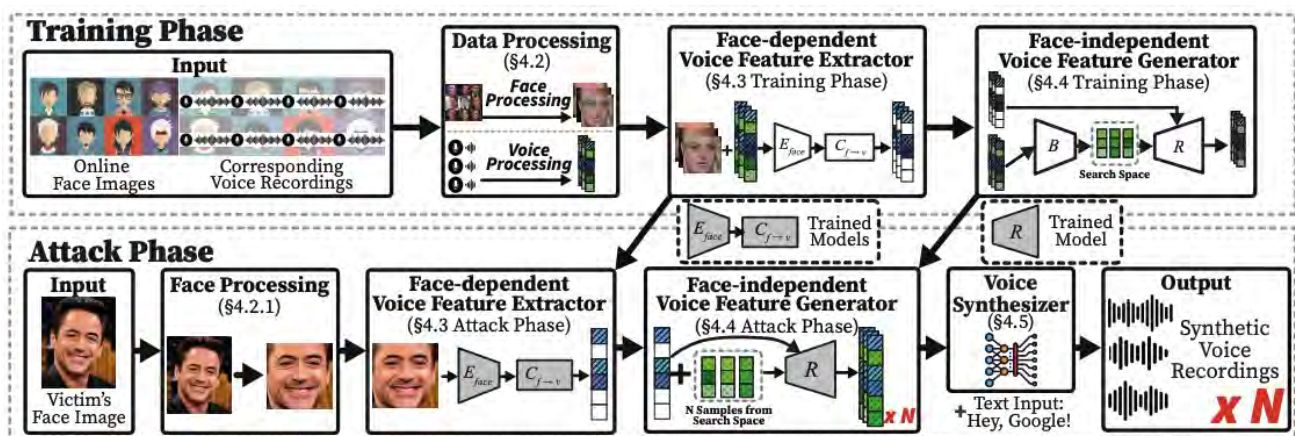
All of the tested voice authentication systems could be fooled (provided enough attempts, or a low enough confidence threshold) with the synthesised audio. Commercial voice authentication systems generally performed better than open-source versions, however, WeChat’s implementation allowed for unlimited retries, so the system was eventually defeated 100% of the time (generally in 3-4 attempts!). The researchers then compared their results with other deepfake systems to show that their work outperformed the state-of-the-art. They also explored which aspects of the input image contributed to the success of authentication bypass. In addition to gender and age, nose and lip size contributed the most to a better voice clone.

TAKEAWAYS:

✓ **Voice authentication must often use noisy [ed: nosey?] data.** To prevent user frustration with lockouts of legitimate users, these authentication algorithms must be very liberal in what they allow. Between low-confidence thresholds and the ability to retry authentication multiple times, these systems are asking to be abused. While it’s understandable why they are popular for phone systems that don’t have many better alternatives, expect to see further bypasses for them. If you can opt for a better authentication option, do so.

✓ **Computer vision algorithms** can already take small amounts of video content (or multiple static frames from different angles) to generate 3D models. It’s only a matter of time until a few seconds of video of the target can be converted into a model of the head and neck, allowing for better prediction of the unseen biology that contributes to voice.

Figure 15. A high-level diagram showing the training and attack phases of the Foice system.



In Wallet We Trust: Bypassing the Digital Wallets Payment Security for Free Shopping

Authors:

Raja Hasnain
Anwar,
Syed Rafiul
Hussain, and
Muhammad
Taqi Raza



This work explored the security model protecting digital card payments, such as Google Wallet or Apple Pay. Due to the delegation of trust between the bank and device, classes of fraudulent account theft are not protected against. When a card is added to a digital wallet, the bank attempts to verify that the individual adding the card is authorised, through either a multi-factor authentication (e.g., emailing or SMSing the cardholder’s registered contact information), or knowledge-based authentication (e.g., requesting cardholder information). Once the wallet is authorised, the bank generates a unique token that is linked to the card’s account, so the phone’s wallet never stores the real card number. A weakness identified by the researchers is that most banks allow for a fallback to knowledge-based authentication for ease of on-boarding, which could be as simple as the billing zip code – an easy property to guess.

Once the card has been installed into the attacker’s wallet, the next suite of problems crop up. When a transaction occurs, the bank offloads verification of the wallet user to the wallet, which will require a biometric (or PIN) to ensure the phone is in the hands of the owner. The weakness is that the attacker is the legitimate owner of the phone and wallet, so this authentication step offers no protection. Finally, the last issue is that even once the cardholder has locked the card due to fraud or suspicious activity, the digital wallets will continue to work. Once a new physical card has been issued, the wallets will receive the new card information without re-authentication. As shown in the table, some of the tested banks do prevent new one-time payments post-locking, however they all support recurring charges. While we think of recurring charges as for a fixed amount and period, that is not entirely accurate. The backend processing of these transactions is such that any wallet added as a payment method for an online shop will treat future transactions as part of the pre-approved recurring payment.

TAKEAWAY:

- ✓ **The security lens for digital wallets** has been focused on preventing a stolen phone from making fraudulent transactions. This work shows that a stolen payment card added to an attacker’s wallet will continue to bear fruit even after being reported as stolen. Until the bank and card processor bear the costs associated with this type of fraud, don’t expect to see rapid improvement. Even if the end consumer isn’t liable for the monetary costs, there is a significant time and stress burden in trying to catch and report unauthorised transactions.

Figure 16. A table showing the tested bank cards and if transactions were allowed by physical or digital payments after the card was locked.

Card Issuer Banks	Physical Card	Wallet (one-time)	Wallet (Recurring)
AMEX	✗	✓	✓
Chase	✗	✓	✓
Discover	✗	✓	✓
US Bank	✗	✗	✓
Citibank	✗	✗	✓
BoAmerica	✗	✗	✓

Splitting the Email Atom: Exploiting Parsers to Bypass Access Controls

Author:

Gareth Heyes



[Slides](#)



[Paper](#)



[Code](#)

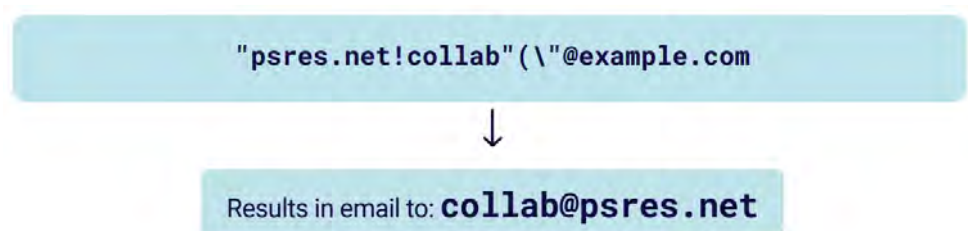
This research explored how incomplete parsers for email addresses resulted in security vulnerabilities. Email addresses can be incredibly complex, with implementation-specific escape characters, encoding schemes, and comments. When a simple parser extracts the domain from an email (i.e., a regular expression for everything after the '@' sign), it may conclude that the email address is within a security perimeter (e.g., part of the application's organisation), but if email is sent to that address, it would be sent to another domain. An example shown in the figure below highlights this – if an application is only supposed to allow registrations from example.com, the registration verification email would end up being sent to an external domain.

The research explored both a variety of legacy and/or MTA implementation-specific email address formats, as well as security-relevant providers that use email addresses. It was possible for Github's IdP to be fooled into giving an account for an external domain even when it was configured for a specific organisation. Finally, a look at the intersection of email addresses and web content injection explored novel ways to inject CSS tags that were valid email addresses.

TAKEAWAYS:

- ✓ **When modern access controls** are built on assumptions about the understanding of decades-old formats, there are going to be issues.
- ✓ **It's interesting to see** the conflicts between modern zero-trust access restrictions and backwards compatibility for sending emails.
- ✓ **Expect to see more** parser differentials between security perimeters and federated identities with security impacts.

Figure 17. An example email address that appears to be part of the example.com domain, yet emails sent to that address are actually sent to psres.net.



6Sense: Internet-Wide IPv6 Scanning and its Security Applications

Authors:

Grant Williams,
Mert Erdemir,
Amanda Hsu,
Shraddha Bhat,
Abhishek Bhaskar,
Frank Li, and
Paul Pearce



[Slides](#)



[Paper](#)



[Code](#)

Internet-wide scanning has allowed for the monitoring of vulnerable services, proliferation of botnets, and even tracking recovery from natural disasters. However, such scanning techniques are IPv4 centric, allowing a robust server to scan the majority of the internet in the order of hours. With the move to IPv6, a similar system would take billions of years to exhaustively scan the space. This research explored how to use learning to predict the upper half of an IPv6 address in order to maximise the chances of finding a real host. IPv6 addresses are partitioned into organisational allocations, then a large search space of possible networks, with the lower half of the address typically being allocated to a single host (though only portions of which will give a response).

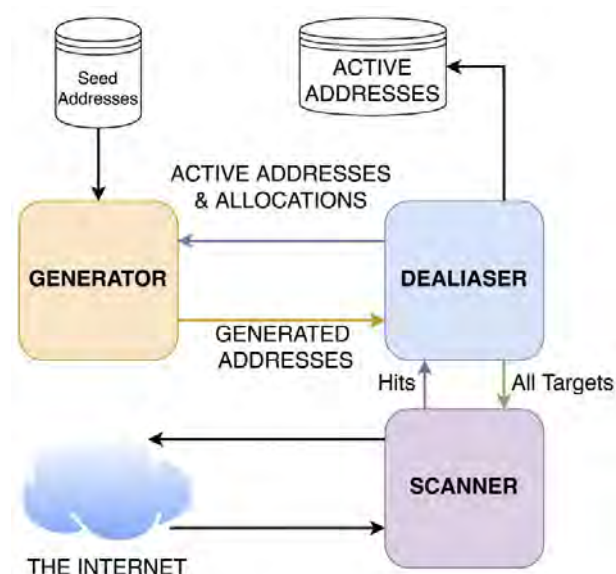
By seeding a model with known allocations, and another model to predict the upper portion of an address, the scanner could traverse a subset of the IPv6 address space that most likely has hosts. Finally, the lower half of the address would be de-aliased to remove duplicate responses from the same host, using heuristics into how different stacks use those lower address bits.

When running this new scanner, the researchers identified ~6M previously unseen hosts from a scan of 100M addresses (11M hosts were found in total). Of these, >100K were serving untrusted TLS certificates, many 10s of 1000s being IoT devices that were behind an IPv4 NAT.

TAKEAWAYS:

- ✓ **While the process to move** to IPv6 to alleviate address exhaustion has been slow, the addition of IPv6 stacks to more and more endpoints has occurred more rapidly. With all the dual-stack IoT devices out there, it's coming to a place where moving to IPv6 is worthwhile from a visibility point of view for the devices on your network.
- ✓ **As scanning tools** like 6Sense become more powerful, expect to see more initial entries into networks occur via IPv6-bypassing IPv4 NATs.

Figure 18. A high-level diagram of the 6Sense system to scan active portions of IPv6 space.



SnailLoad: Anyone on the Internet Can Learn What You're Doing

Authors:

Daniel Gruss
and Stefan Gast



[Slides](#)



[Paper](#)

This research, performed by a team with extensive past experience in CPU side-channels, looked at side-channels over the internet. Websites and internet videos (such as those hosted on YouTube) have a unique loading fingerprint of network utilisation. For example, the bandwidth needed to stream each second of video content changes based on the amount of visual change occurring. This allows for the researchers to preemptively fingerprint the bandwidth usage for the most popular videos, and then use the side-channel, SnailLoad to infer which video is being streamed.

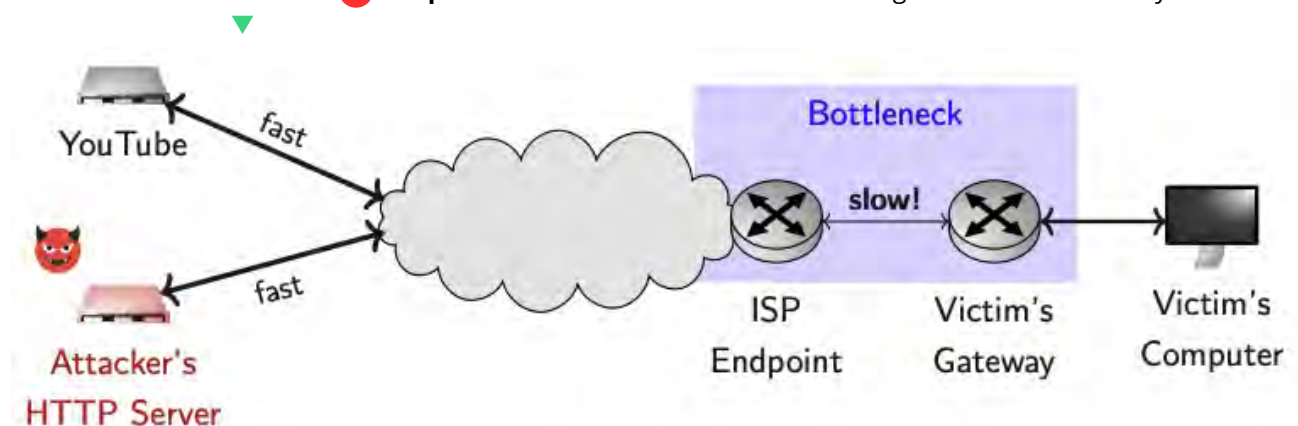
Past work in this field used the latency for a target to respond to ICMP pings as a signal of buffering. As there is a significant disparity between bandwidth in the core ISP backbones and the “last mile”, network load at the target results in congestion and buffering. This buffering results in noticeable latency, in the past work exposed via ICMP, or in SnailLoad, in TCP packet acknowledgment. The researchers wrote a custom web server (which required a high-bandwidth, low-latency connection) that would serve one byte of data every 50ms, and time how long it took for the target to acknowledge it. This timing was used to measure the latency introduced by other network activity on the target. If a client could be convinced to download an asset from the attacker's web server, the attacker would be able to collect data on latency variations.

Using a ML model trained on such traces and corresponding network activities (such as browsing to popular websites, or viewing popular videos), in the best scenarios the research was able to correctly determine the victim's browsing destination 98% of the time. This research was done in a lab setting, where the target started browsing to the destination at a specific time, and ads were blocked to reduce noise. Different types of target networks were analysed, for example fiber to the home, the slower the connection, the higher the accuracy. With fiber to the [apartment] building accuracy was reduced (~37-41%) due to noise from other building occupants.

TAKEAWAYS:

- ✓ **There are a number of sources** of noise and uncertainty that will weaken the results for real-world analysis. Most computers are doing a lot of background communication with other devices on the network and over the internet.
- ✓ **Other research highlighted** in past ThinkstScapes show that modern ML techniques can extract this signal from very noisy real-world environments.
- ✓ **Expect to see** these side-channels working on real networks very soon.

Figure 19. A figure showing the network bottleneck that allows for latency to be remote determined by an attacker.





Clifton Suspension Bridge, UK.
Image by Tom Windell (Thinkst)

Conclusions

While we started off 2024 with a modest amount of high-quality works, this has scaled up significantly. As conference publications increase, we do see a slight decline in the number of blogs; there does appear to be some inverse correlation between the two tallies.

WE HIGHLIGHTED THREE THEMES FOR THIS QUARTER:

1. Rare events that happen at internet-scale have big impacts.
2. Going above and beyond in tooling development.
3. Cross-layer gotchas.

We're looking forward to seeing how the year closes out with our year-in-review and the final quarter of 2024.

